TP 00 — Premiers pas en Python

Programmer le déplacement d'une voiture en 15 minutes

Sixtine Michel

De Vinci Higher Education — ESILV

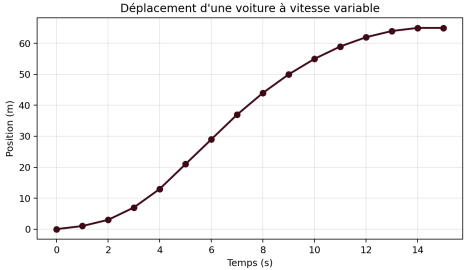
ESILV
DE VINCI
ENGINEERING
SCHOOL

Notre objectif

La vitesse d'une voiture augmente, reste élevée, puis diminue.

En quinze minutes

Nous allons utiliser des variables, un calcul, une boucle, des listes et un premier graphique.



D'un point de vue algorithmique

Le programme est découpé en trois fonctions appelées dans cet ordre :

```
def simuler_voiture():
    # Calculer les positions et les distances.

def afficher_resultat(temps, positions):
    # Tracer la position au cours du temps.

def main():
    # Simuler, puis afficher le resultat.
```

Chaque fonction réalise une tâche précise et peut être testée séparément.

Variables et calculs

```
1 DT = 1.0
2 VITESSES = [0, 1, 2, 4, 6, 8, 8, 8, 7, ..., 0]
3 position = 0.0
```

Une variable possède un nom et une valeur. Le signe = remplace la valeur située à gauche par le résultat calculé à droite.

Répéter avec une boucle — TODO 1

```
1 for etape in range(0, NB_ETAPES + 1):
2     # instructions repetees
```

`range(0, 16)` produit les entiers de 0 à 15, comme les indices de `VITESSES`. Les instructions indentées appartiennent à la boucle.

Lire la vitesse — TODO 2

```
1 vitesse = VITESSES[etape]
```

L'indice sélectionne la vitesse de l'étape courante. Par exemple, `VITESSES[5]` vaut 8 m/s.

Calculer le déplacement — TODO 3

```
deplacement = vitesse * DT
position = position + deplacement
```

Les unités donnent $\mathrm{m/s} \times \mathrm{s} = \mathrm{m}$. La nouvelle position remplace ensuite l'ancienne.

Mémoriser dans des listes — TODO 4

```
temps.append(etape * DT)
positions.append(position)
```

append ajoute une nouvelle valeur à la fin d'une liste. Les deux listes gardent ainsi tout l'historique du mouvement.

Calculer une distance — TODO 5

En une dimension, la distance entre deux points a et b est :

$$d(a, b) = \sqrt{(a - b)^2}.$$

```
distance_origine = sqrt(position ** 2)
distance_point = sqrt(
    (position - POINT_REFERENCE) ** 2)
print(etape, position, vitesse,
      distance_origine, distance_point)
```

Le carré rend la différence positive et la racine carrée redonne une distance. Le tableau affiche ces deux distances à chaque étape. À la fin, elles valent 65 m depuis l'origine et 15 m depuis le point placé à 50 m.

Tracer la courbe — TODO 6

```
plt.plot(temps, positions, marker="o", linewidth=2)
plt.show(block=True)
```

La première liste fournit les abscisses et la seconde les ordonnées. Dans Spyder, utiliser le backend **Automatic** ou **Qt**.

À retenir

- variable et affectation ;
- calcul arithmétique ;
- carré et racine carrée ;
- boucle `for` ;
- indentation ;
- liste et `append` ;
- courbe Matplotlib ;
- lecture d'un résultat ;
- relation distance–vitesse–temps ;

Vous êtes prêts pour le projet !